Wait Less
Detailed Project Stage Report

Sapienza IoT Project Team

April 8, 2026

Contents

1. Executive Summary

Wait Less is a queue-aware adaptive traffic light project for a two-way intersection built around two ESP32 LoRa nodes, two sensing lanes, and six traffic-light LEDs. The system is designed to sense incoming traffic and queue presence on both sides of the intersection, exchange compact telemetry between nodes, and adapt the green time based on current demand instead of relying on a fixed cycle.

At the current stage of development, the project has moved from a concept description into a structured technical repository with a shared controller, firmware skeletons for both boards, project documentation, and a local simulator that already demonstrates adaptive behavior. This report documents the implementation stages from the initial technical foundation to the expected final delivery, with special attention to the checkpoint delivery scheduled for Friday, April 10, 2026.

2. Project Objective

The main objective of the project is to build an IoT-based traffic-light prototype that:

- detects traffic on two sides of an intersection,
- estimates traffic demand using near and far sensors,
- exchanges lightweight telemetry using LoRa,
- and dynamically assigns green light time to the side with higher demand.

The system is intentionally designed as a distributed embedded system rather than a single centralized microcontroller. This allows the project to demonstrate multiple IoT concepts at once: sensing, embedded processing, communication, control logic, and evaluation under real-time constraints.

3. System Overview

3.1. High-Level Architecture

The system is organized around two nodes:

- **Node A**: reads the sensors for side A and transmits telemetry over LoRa.
- **Node B**: reads the sensors for side B, receives telemetry from Node A, executes the adaptive controller, and drives the six LEDs representing the traffic lights.

Each side uses two sensors:

- one far sensor to detect approaching vehicles,
- one near sensor to detect queue presence close to the stop line.

The traffic controller uses these measurements to estimate demand and decide whether to keep the current green phase or switch to the other side.

3.2. Current Repository Structure

The current implementation is organized as follows:

- `README.md`: project summary and usage notes,
- `platformio.ini`: PlatformIO environments for the two ESP32 nodes,
- `lib/traffic_control/`: shared traffic types, sensing, messaging, and controller logic,
- `firmware/node_a/main.cpp`: sensing firmware for side A,
- `firmware/node_b/main.cpp`: sensing and controller firmware for side B,
- `simulation/simulate_traffic.py`: local simulation of the adaptive controller,
- `docs/checkpoint-demo.md`: presentation and demo outline,
- `docs/project-roadmap.md`: stage-based development roadmap.

3.3. Current Control Logic

The shared controller currently follows a simple and safe adaptive strategy:

1. keep a minimum green duration to avoid unstable rapid switching,
2. switch if the current green side becomes empty and the other side has demand,
3. switch after the minimum green if the other side is clearly busier,
4. force a switch after a maximum green time if the other side is still waiting.

The queue-aware demand score combines queue estimate, far-sensor occupancy, and near-sensor occupancy. This makes the controller more robust than a binary vehicle detector because it can distinguish between a lane that is only momentarily occupied and a lane that is building pressure.

3.4. Telemetry Format

The current implementation uses a compact CSV-style message:

```
A,1,0,4,2,2,0,12345
```

This example represents:

- side A,
- far sensor occupied,
- near sensor not occupied,
- incoming count equal to 4,
- passed count equal to 2,
- estimated queue equal to 2,
- emergency request equal to 0,
- timestamp equal to 12345 ms.

4. Current Technical Progress

4.1. Implemented Assets

The following technical assets have already been created:

- a shared embedded controller implementation in `AdaptiveController.cpp`,
- a queue estimator in `TrafficSensing.cpp`,
- telemetry encoding and decoding in `NodeMessaging.cpp`,
- firmware entry points for both nodes,
- a simulation script that demonstrates controller behavior over time,
- documentation for the checkpoint demo and the full roadmap.

4.2. Simulation Evidence

The local simulation has already been executed successfully and shows the expected behavior: the controller starts with side A green, keeps it while side A is busy, schedules a yellow phase when side B becomes clearly busier, and then switches to side B.

An excerpt of the simulation output is shown below:

```
time | queueA | queueB | green | phase | currentDemand | otherDemand
   0s | 3 | 0 | A | GREEN | 15 | 0
  12s | 1 | 4 | A | YELLOW | 9 | 18
  14s | 1 | 4 | B | GREEN | 18 | 9
```

This result is important for the checkpoint because it provides a valid technical demo even before the full hardware integration is complete.

4.3. Checkpoint Baseline Parameters

For the April 10 checkpoint, the team should present the current implementation baseline using explicit numerical values already defined in the repository:

- control-loop interval: `200 ms`,
- telemetry transmission interval: `1000 ms`,
- far ultrasonic threshold: `35 cm`,
- near ultrasonic threshold: `18 cm`,
- minimum green time: `5000 ms`,
- yellow time: `2000 ms`,
- maximum green time: `20000 ms`,
- switching margin: `4` demand-score points,
- demand-score model: `3 * queue + 2 * farOccupied + 4 * nearOccupied`,
- example telemetry payload: `A,1,0,4,2,2,0,12345`, which is `19 bytes`.

These values provide a concrete baseline for the review. They are more defensible than qualitative claims because they correspond directly to the current source code.

4.4. Checkpoint Metrics And Experimental Evidence

The checkpoint should focus on the metrics that can already be justified from the current code and simulation baseline:

- controller update period,
- telemetry update period,
- telemetry payload size,
- response time from a traffic change to yellow start,
- response time from yellow start to new green,
- upper bound on waiting time enforced by the maximum green cap.

The following scripted controller experiments have already been executed on the current baseline:

1. **Equal demand should not trigger a switch.** From `t = 6 s` to `t = 11 s`, both sides have queue `2`. The controller keeps side A green during the full interval. This confirms that equal demand does not create unnecessary oscillation.
2. **A busier opposing side should trigger a switch after the minimum green is satisfied.** At `t = 12 s`, side A queue becomes `1` and side B queue becomes `4`. Yellow starts at `12 s` and side B becomes green at `14 s`. This confirms that the controller reacts immediately at the decision level while preserving the `2000 ms` yellow interval.
3. **If the current green side becomes empty while the other side has demand, the controller should yield at the first allowed instant.** In a scripted test where side A becomes empty at `t = 6 s` and side B queue becomes `2`, yellow starts at `6 s` and side B becomes green at `8 s`. This confirms that empty-lane detection is connected correctly to the switching logic.
4. **The maximum green cap should prevent starvation.** In a scripted test where side A queue remains `3` and side B queue remains `1`, yellow starts at `20 s` and side B becomes green at `22 s`. This confirms that one side cannot keep green indefinitely while the other side still has demand.

These experiments do not yet replace hardware evaluation, but they are sufficient to demonstrate that the decision logic already follows a measurable and testable process.

5. Stage Summary

Stage	Main Goal	Status	Main Output
0	Align the project and create the initial technical foundation	Completed	Repository, shared logic, simulator, documentation
1	Prepare and deliver the April 10 checkpoint presentation and demo	In progress	Slides, demo, video, repository package

Stage	Main Goal	Status	Main Output
2	Bring up the sensing hardware and validate measurements	Planned	Calibrated sensors and verified LED states
3	Integrate real LoRa communication between nodes	Planned	Packet exchange and communication logs
4	Connect live sensing to the adaptive traffic-light controller	Planned	End-to-end adaptive prototype
5	Evaluate performance and improve the system	Planned	Metrics, logs, and refined parameters
6	Package the final submission and final demo	Planned	Final repository, report, slides, and video

6. Detailed Stage Report

6.1. Stage 0: Project Alignment and Technical Foundation

Goal. The purpose of Stage 0 was to convert the original idea into a structured technical project that could be implemented, demonstrated, and extended.

Work completed.

- The project concept was consolidated into a single technical direction based on two ESP32 LoRa nodes and queue-aware traffic-light control.
- The repository was structured so that code, documentation, and demo material are separated clearly.
- A shared traffic-control library was created to avoid duplicating decision logic across firmware files.
- A sensing model was added to track incoming traffic, passed vehicles, and estimated queue size.
- A local simulator was created to demonstrate traffic scenarios without depending on full hardware readiness.

Why this stage matters. Without this stage, the project would still exist only as a presentation concept. Completing Stage 0 created a real technical baseline on top of which the remaining work can be carried out in a disciplined way.

Deliverables already produced.

- `README.md`
- `platformio.ini`
- `lib/traffic_control/*`
- `firmware/node_a/main.cpp`
- `firmware/node_b/main.cpp`
- `simulation/simulate_traffic.py`
- `docs/checkpoint-demo.md`

- `docs/project-roadmap.md`

Status. Completed.

6.2. Stage 1: Checkpoint Delivery on April 10, 2026

Goal. The purpose of Stage 1 is to demonstrate clear technical progress during the 5-minute checkpoint delivery, using explicit requirements, measurable metrics, and concrete experiments rather than only architecture diagrams.

Required outputs for this stage.

- a short technical presentation,
- a checkpoint slide that states the current baseline requirements with explicit numbers,
- a checkpoint slide that defines the metrics used for verification,
- a checkpoint slide that summarizes experiments, results, and conclusions,
- a clear demo of at least one core functionality,
- early evaluation of the main components,
- a public YouTube video of the demo,
- and a GitHub repository containing all the material.

Material already available for the checkpoint.

- A project overview and technical structure are already documented.
- The simulator provides a credible demo of adaptive control behavior.
- The firmware skeleton already expresses the node roles and data flow.
- The baseline controller parameters already provide numeric values for the checkpoint discussion.
- The roadmap and checkpoint notes already describe the remaining work.

Remaining work before the checkpoint.

- prepare or refine the presentation slides around requirements, metrics, and experiments,
- record a short demo video,
- create a public repository and upload the material,
- summarize the current experiments and results in a compact table,
- align all documentation on the exact hardware choice.

Recommended checkpoint demo.

1. Explain the architecture in one slide.
2. Present the baseline requirements with explicit numbers.
3. Present the metrics and the current experiment results.

4. Run the simulator and explain one switching event from demand change to yellow to green.
5. If possible, add a short bench clip with LEDs or manually emulated sensor input.
6. End with the development plan for the next stages.

Status. In progress.

6.3. Stage 2: Hardware Bring-Up and Sensor Validation

Goal. Stage 2 focuses on making the sensing hardware reliable on the real boards.

Planned work.

- Wire the far and near sensors for both sides.
- Confirm the final sensor technology used in the project documentation.
- Calibrate thresholds for lane occupancy and queue detection.
- Validate that the six LED outputs match the expected traffic-light states.

Expected results. At the end of this stage, the team should know whether the sensors are stable enough for repeated testing and what threshold values should be used in the firmware.

Main risk. If the sensor readings are noisy or inconsistent, the controller may switch at the wrong time. This stage is therefore a prerequisite for trustworthy integrated testing.

Status. Planned.

6.4. Stage 3: LoRa Communication Integration

Goal. Replace the current communication stub with real LoRa transmission between the two nodes.

Planned work.

- Integrate the chosen LoRa library into the firmware.
- Send telemetry from Node A to Node B.
- Decode and validate the received telemetry on Node B.
- Measure message timing, reliability, and the effect of packet loss.

Expected results. At the end of this stage, the system should support real distributed communication instead of local emulation.

Main risk. Communication failures may create stale data at the controller. This means the firmware should be designed to tolerate an occasional missed packet and continue safely.

Status. Planned.

6.5. Stage 4: Full Adaptive Traffic-Light Control

Goal. Connect real sensor input and real communication to the controller and drive the physical traffic-light LEDs accordingly.

Planned work.

- Use live telemetry from both sides as controller input.
- Apply minimum green, yellow, and maximum green timing rules.

- Validate that the controller switches at appropriate times.
- Test the behavior under empty-lane, balanced, and asymmetric demand scenarios.

Expected results. This stage should produce the first full end-to-end prototype that behaves adaptively in real time.

Main risk. Even if sensing and communication work separately, timing interactions may still create instability. Bench testing with controlled scenarios is essential.

Status. Planned.

6.6. Stage 5: Evaluation and System Improvement

Goal. Produce measurable evidence about system behavior and improve the design where necessary.

Planned work.

- Measure responsiveness and latency.
- Estimate or measure average node energy usage.
- Log representative traffic scenarios.
- Compare the adaptive strategy to a simple fixed-cycle baseline.
- Tune thresholds, timing values, and transmission intervals.

Expected results. By the end of this stage, the project should not only work, but also provide evidence that the design decisions are technically justified.

Main risk. A working prototype without evaluation is difficult to defend academically. This stage turns implementation into engineering evidence.

Status. Planned.

6.7. Stage 6: Final Delivery and Project Packaging

Goal. Prepare the final complete submission with coherent documentation, final code, final demo material, and a clear explanation of the system.

Planned work.

- Finalize the repository structure and documentation.
- Prepare the final slides and video demo.
- Summarize the architecture, implementation choices, results, and lessons learned.
- Ensure that all required submission links and artifacts are available in the same repository.

Expected results. The final stage should produce a complete and reviewable project package, not only a collection of source files.

Main risk. If packaging is left too late, the project may be technically strong but difficult to evaluate. This stage ensures the work is communicated professionally.

Status. Planned.

7. Immediate Next Actions

The most important next actions for the team are the following:

1. complete the checkpoint presentation for April 10, 2026,
2. organize the presentation around requirements, metrics, and experiments,
3. record the simulator demo and, if possible, a short hardware clip,
4. publish the project in a GitHub repository,
5. keep the hardware documentation consistent across the report, slides, and code,
6. begin Stage 2 hardware validation immediately after the checkpoint.

8. Conclusion

The project has already completed the most important transition from concept to implementation baseline. The repository now contains working documentation, an adaptive controller, firmware scaffolding for both nodes, and a simulation-based demo that can support the checkpoint presentation. The next step is not to redesign the project, but to continue integration in an orderly way: first the checkpoint delivery, then hardware validation, communication, full integration, evaluation, and final packaging.

This staged approach reduces risk and gives the team a clear path from the current checkpoint to the final delivery. It also ensures that each technical component of the system is implemented, tested, and documented before the project is presented as complete.